

# 1 Renderer 2

## 2 Indirect Rendering

In the new renderer, the main passes have been structured around gpu-driven rendering. The current design uses the traditional geometry rasterization pipeline (see Section 2.3), but dispatches draw-calls from the GPU instead. This allows for more flexible rendering and controlling draws from GPU.

The biggest goal in this design was to support culling and LOD selection on the GPU, reducing CPU overhead.

Pipeline changes and state changes are conventionally not supported directly on the GPU. But draw calls can be produced by using `VkDrawIndexedIndirect`. This reads draw calls from a GPU buffer, meaning that only a single `VkDrawIndexedIndirect` command from the CPU needs to be submitted.

To support this, the renderer needs to reduce state changes and pipeline changes. This is done by

- Using a single vertex and index buffer
- Splitting objects into batches (a batch is a group of objects that share the same pipeline)

To support this, a mirror of the scene state needs to be present at all times on the GPU. We call this the *object buffer*. An object keeps reference to its mesh, its material and its transformation. When the scene changes on the CPU, the necessary updates need to happen on the GPU as well.

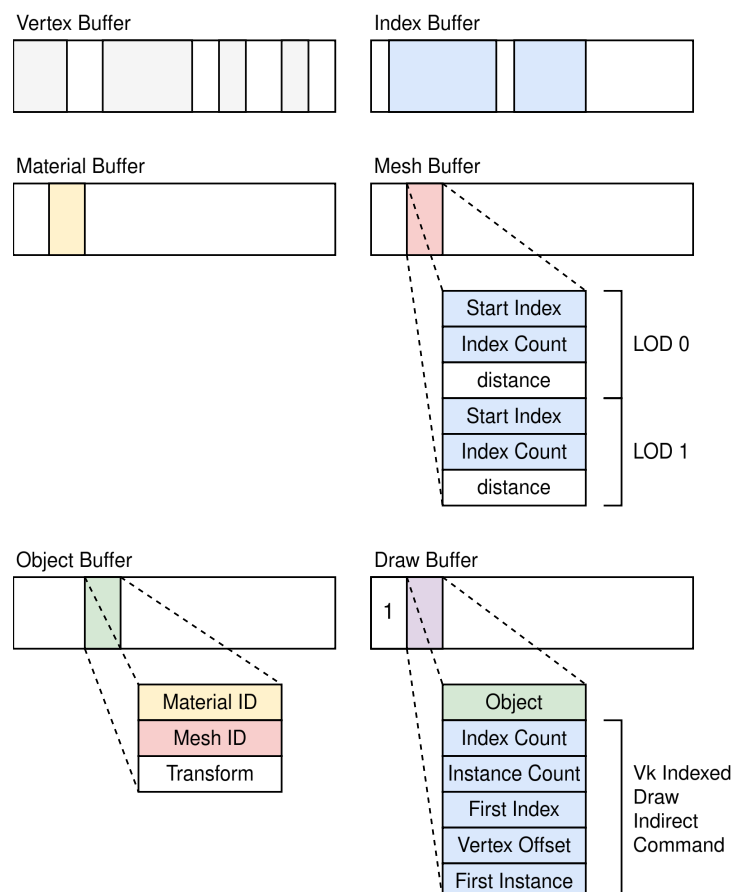


Figure 1: Indirect Rendering

### 2.1 The Mesh Pass

We call a full draw of the scene a *mesh pass*. There can be multiple mesh passes, like the main forward pass, a directional shadow pass, etc. A mesh pass is first an invocation of the culling

compute shader, generating elements in the *draw buffer*. The draw buffer is later consumed by the *forward indirect* pass, which performs one *VkDrawIndexedIndirect* per batch.

## 2.2 Manual Vertex Pulling

A problem with the current design is that a batch requires all object to use the same vertex layout. While not an issue for us, it is a limitation that could be avoided. The whole Input Assembly stage can be avoided by manually fetching vertex data in the vertex shader. Instead of using a vertex buffer, the vertices can be passed in a storage buffer, and the batches just store the index into it. Each batch specifies its own stride.

This should be trivial to implement, and would allow for more flexible rendering.

## 2.3 A note on Mesh Rendering

Mesh rendering replaces the old rasterization pipeline, and replaces Input Assembler and Vertex stage with a compute-like shader. This shader can directly output to the rasterizer. A feature of mesh shading is that the mesh shader directly works in workgroups, meaning that it is critical for the application to split meshes into smaller parts (64/128 triangles) called *meshlets*. Culling can be done cheaper than an AABB frustum test if the meshlets are constructed convexly.

Using mesh rendering instead of the current system would have been simpler and likely more efficient. We wouldn't need the current draw buffer. Culling could be implemented on a meshlet level instead of objects, meaning that the geometry would be more evenly distributed.

Overall, mesh shading would be really interesting to try out.